

A SaaS app with zero app code

formal guardrails for LLM agents, act III: the developer experience

Flowdeck — a multi-tenant team kanban — built end to end as one rule base:
7 tickets, 377 lines of YAML, a clickable web UI, and a solver reviewing every change.

With screenshots of the running product, the unedited development journal (two real authorization holes caught), and what this way of working actually feels like. August 2026.

No formal-methods background assumed — the four concepts you need are on slide 3.

Agents now out-write our ability to check

- LLM agents produce code faster than humans can meaningfully review it. Human attention is the bottleneck — and it samples, it does not cover.
- Tests also sample: they check the runs you thought of. The bugs that matter live in the interactions you did not think of.
- If we want agents (and background jobs, and time) to act autonomously, we need a way to say what must never break — and have a machine enforce it.

The proposal

The developer writes down intent as rules and invariants. Agents write the rest. Reasoning engines sit between them and arbitrate: every change must provably keep the rules.

Code becomes cheap to regenerate. The spec becomes the durable asset.

Setting: a regular web SaaS application — CRUD/API handlers, authorization, tenancy, workflows, background jobs. Not kernels, crypto, or avionics: every technique here is judged against ordinary product teams, CI budgets, and mainstream languages at the edges.

All the formal methods you need today

Invariant

A sentence about the system that must be true at every moment. Example: “a user never sees another team's data.” You already write these — in comments, runbooks and post-mortems. Here they become machine-checkable.

Model

A small, faithful board-game version of your system: its states and legal moves. Here the “model” is not beside the system — the rule base IS both the model and the running program.

Checker

A tireless adversary (here: the Z3 solver). It considers every situation the rules allow — tens of thousands you would never test — hunting for one that breaks an invariant.

Counterexample

The checker's proof of failure: the exact situation that breaks the rule, with the rule that allowed it named. Not “something is wrong” but “an anonymous visitor assigned to a task can start it, via assignee_moves.”

Proofs beside the code: it works — and it has a tax

2 protocols falsified, then proven

P1: the model checker beat a careful engineer twice on an online DB migration. Version 3 survived — and was later proven inductively, for any system size, and live (tracks I, J, M).

59 anomalies per run

P3: skip the guard against real Postgres under concurrent load and the model's predicted anomaly appears 59×/run; 0 with the proven choreography.

1 round to the full fix

P4: same bug, same model, same generic prompt — a stronger frozen gate turned a partial fix into the full fix. Gate strength beats prompt steering.

3.94× faster, never wrong

P5: an agent optimized the CMS freely inside the frozen gate; two correctness-breaking “optimizations” were absorbed mechanically.

Proofs at five levels

Inductive invariants, parameterized (UPDR), liveness under fairness, noninterference via self-composition, a Dafny proof of real session code (tracks I–M).

The residual tax

Every result above polices the model↔code boundary: MBT, trace validation, extraction fidelity. Act II removed that boundary for the ruled part of the system.

Language scouting (notes 11–12): of five ways proofs can meet code, “verified engine + small analyzable DSL” — Cedar's shape — ranked first. Acts II and III build exactly that.

The rule base is the program; the solver is the reviewer

- One artifact: rules.yaml — roles, lifecycle, allow/deny rules with their ticket sentences. The engine serving it is domain-free; refusals are 403s that NAME the rule.
- One condition grammar, two backends: the analyzer's Z3 view and the server's runtime view are the same parsed rules — agreement checked exhaustively, so spec↔code drift is closed by construction.
- The gate is frozen for agents: safety (EVERY situation) + possibility (SOME — “the safest system does nothing” fails) + feature runs with refusals expected by name + gated lifecycle entries.

```
$ analyze rulesets/cms-buggy --gate rulesets/cms # sabotage
FAIL DEAD allow rule 'editor_read_all' - silently masked
FAIL S2_separation_of_duties counterexample + granting rule
FAIL P2_review_by_non_author: IMPOSSIBLE - strict_privacy
VERDICT: FAIL (7 findings) # 4 detector kinds fire at once
```

Evidence so far (notes 13–14)

- Sabotaged CMS: 7 named findings; naive import extension: 5.
- Background jobs are just actors — a compromised importer is contained by policy, provably.
- Domain transfer (receivables): the engine was missing time, not receivables — ~100 generic lines.
- Two redundant rules proven dead and deleted: the solver shrinks rule bases, reversing 1980s rule rot.

Act III asks the question that decides adoption: what does building a real product this way FEEL like — with a UI you can click, and an honest journal of every finding?

Flowdeck: 7 tickets, 377 lines of YAML, zero app code

A multi-tenant team kanban SaaS, developed exactly as the method prescribes: tickets → rules → frozen gate, with every analyzer round journaled (DEVLOG.md) and predictions recorded before the runs that test them.

```
# TICKETS.md — the product spec (excerpts)
TB-1  A team's board is completely invisible to
      every other team. Staff sees everything
      but never decides reviews.
TB-3  Only the assignee starts/submits a task;
      no work starts without an estimate.
TB-4  Nobody approves a task they did themselves.
TB-6  A janitor bot archives stale done tasks —
      and must be able to do nothing else.
```

```
# rules.yaml — the ticket, translated (excerpt)
- id: team_walls
  description: "TB-1: members and leads only
    ever act inside their own team — reading
    included, creating included."
  effect: deny
  when: 'actor.role in ["member", "lead"]
    and not resource.same_team'
```

- The whole application: rules.yaml (160 lines, 18 rules) + safety.yaml (13 $\forall$ -properties, 8 $\exists$ -witnesses) + features.yaml (5 scenarios, refusals by name) + a seed list. UI, API, storage, analysis: inherited, generic.
- New vocabulary this domain forced into the generic engine (~60 lines): actor attributes and actor↔resource relations — same_team, assigned_to_me — as projected booleans. Third domain in a row to cost exactly one missing concept (receivables: time).

Four analyzer rounds, 0.18 s each — all findings real

1

unknown variable 'actor.is_assignee' — typo = load error, named. Prediction “the containment deny is dead” FALSIFIED: an unguarded allow made it load-bearing.

2

TWO REAL AUTHORIZATION HOLES (below) + two probes expecting the wrong deny name (overlapping denies are declaration-order-named).

3

DEAD deny rule 'janitor_scope' — after the fix, default-deny + tight allows contain the bot; gate S11 proves it universally. Deleted: the solver shrinks the rule base.

4

PASS (0 findings). Tickets → green gate: ~1 hour of author time, almost all of it writing YAML, none of it debugging.

34,560 situations

0.18 s / round

2 real holes

~1 h to green

```
$ analyze rulesets/taskboard      # round 2, 0.18 s
```

```
FAIL S2_no_anonymous_access
```

```
  counterexample: {role: anonymous, action: start,
    assigned_to_me: true, ...}
```

```
  allowed by: assignee_moves
```

```
FAIL S5_assignee_works
```

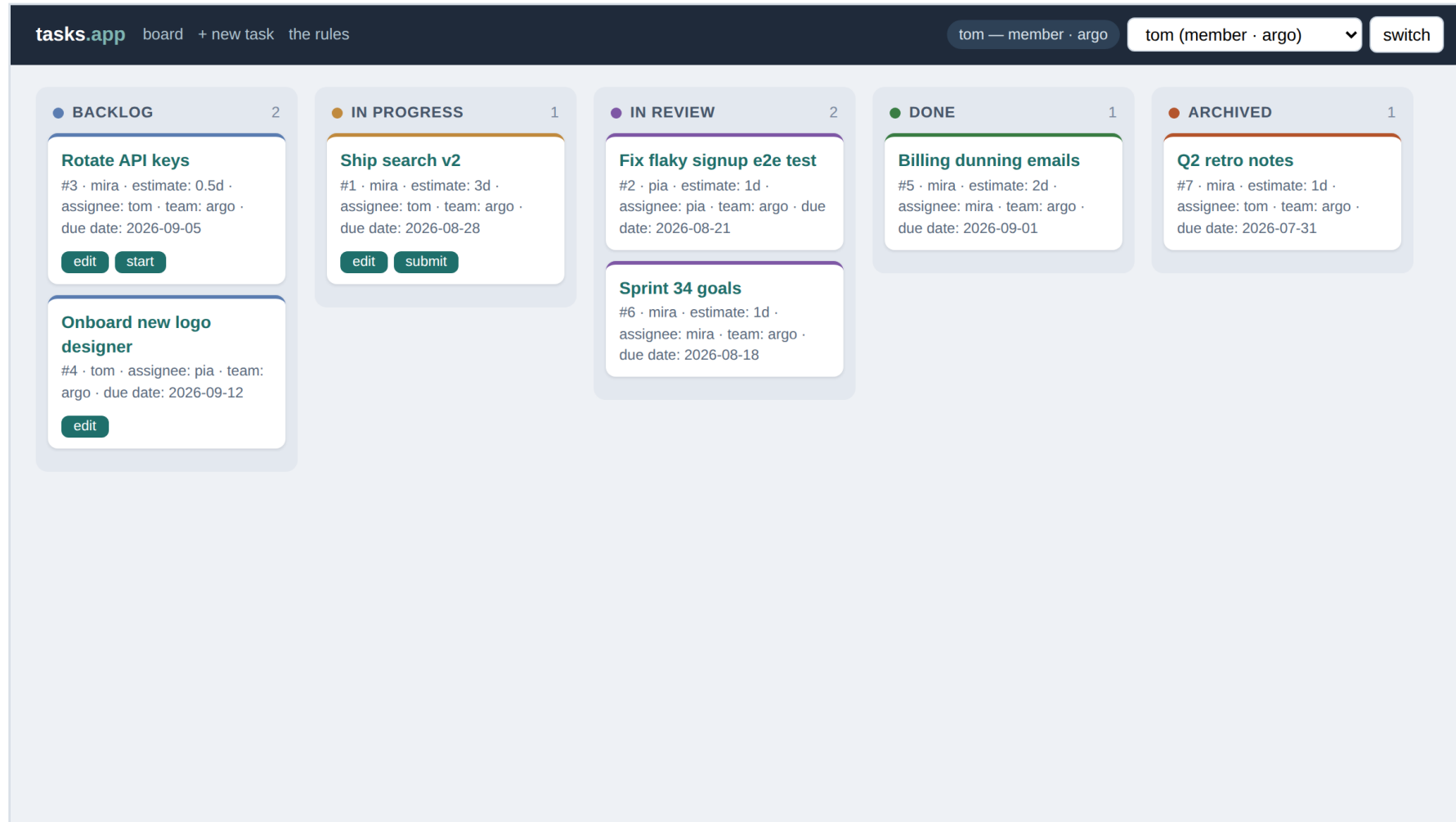
```
  counterexample: {role: admin, action: submit,
    assigned_to_me: false, ...}
```

```
  allowed by: admin_oversees
```

Hole 1: anyone who can write the assignee field could hand work to the anonymous public. Hole 2: “staff can support everything” quietly granted staff the team's work (TB-3 violation). Both are interaction bugs — each rule read fine alone. Nobody writes the test where the task is assigned to the anonymous user; the $\forall$ -check considers all 34,560 situations, then names the granting rule.

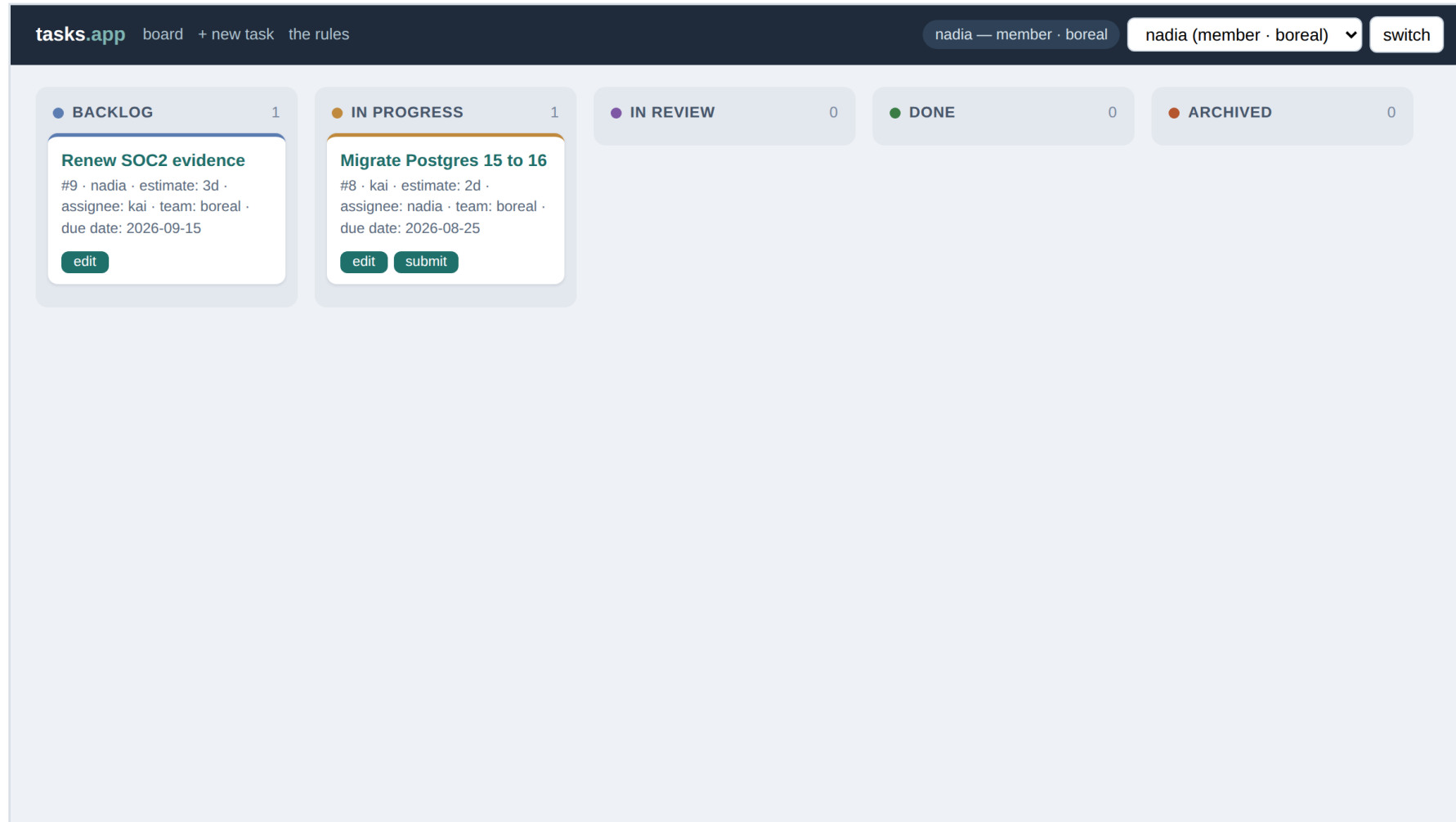
The round-2 draft is preserved (rulesets/taskboard-round2) and held to the frozen gate by check.sh forever — development history as a permanent regression test.

The app, derived — nobody wrote this UI



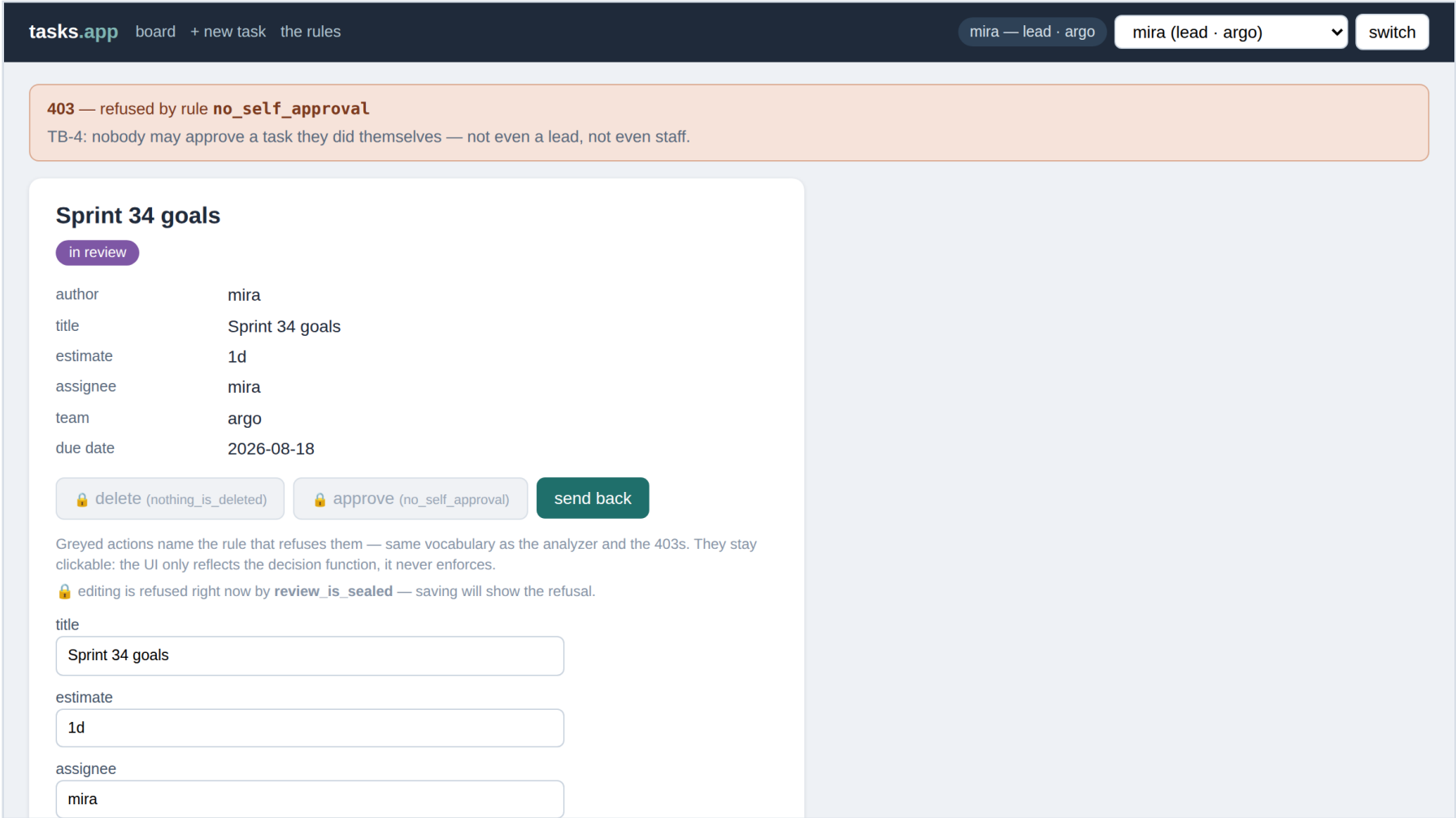
Board columns = the lifecycle table; cards = the read rule; buttons = the decision function (tom gets “start” only where he is the assignee). Persona switcher: authn is out of scope, authz is the study.
python app.py → http://127.0.0.1:8800/ui — the same generic module renders any rule base. The demo data was seeded through the rules over HTTP: a seed that violates policy cannot exist.

Same URL, other team: nadia sees only boreal



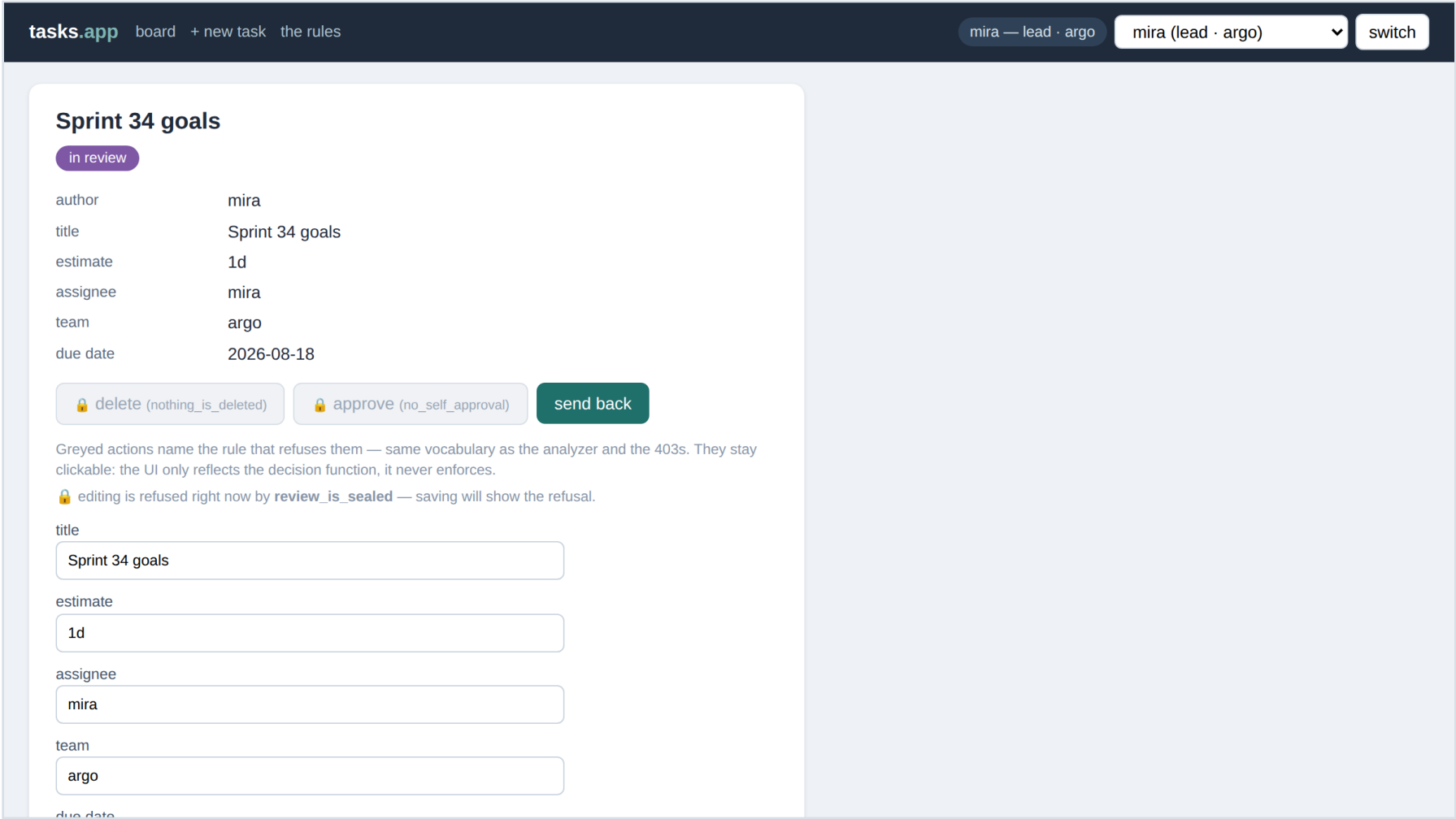
team_walls (TB-1) is one deny rule — and S1_team_isolation proves that no allowed situation lets a member or lead act cross-team. The board IS the read rule, so the proof is what you are looking at. The visibility probe compares the live list endpoint against the decision function's prediction per persona (check.sh stage 4).

mira clicks “approve” on her own task



403 — refused by rule no_self_approval, with the TB-4 ticket sentence. One vocabulary carries from the ticket to the rule id to the solver's counterexample to the button tooltip to this banner. The UI never enforces — denied buttons stay clickable on purpose, and the decision function refuses by name. Locked buttons: delete (nothing_is_deleted), approve (no_self_approval); send back is live.

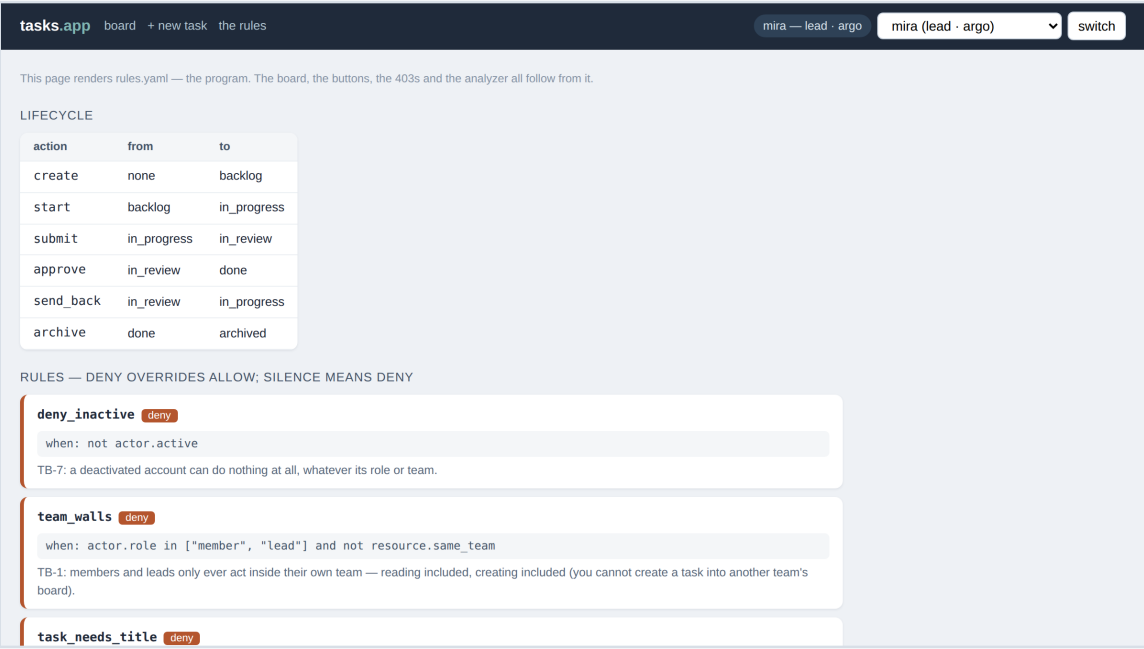
Every control names the rule that governs it



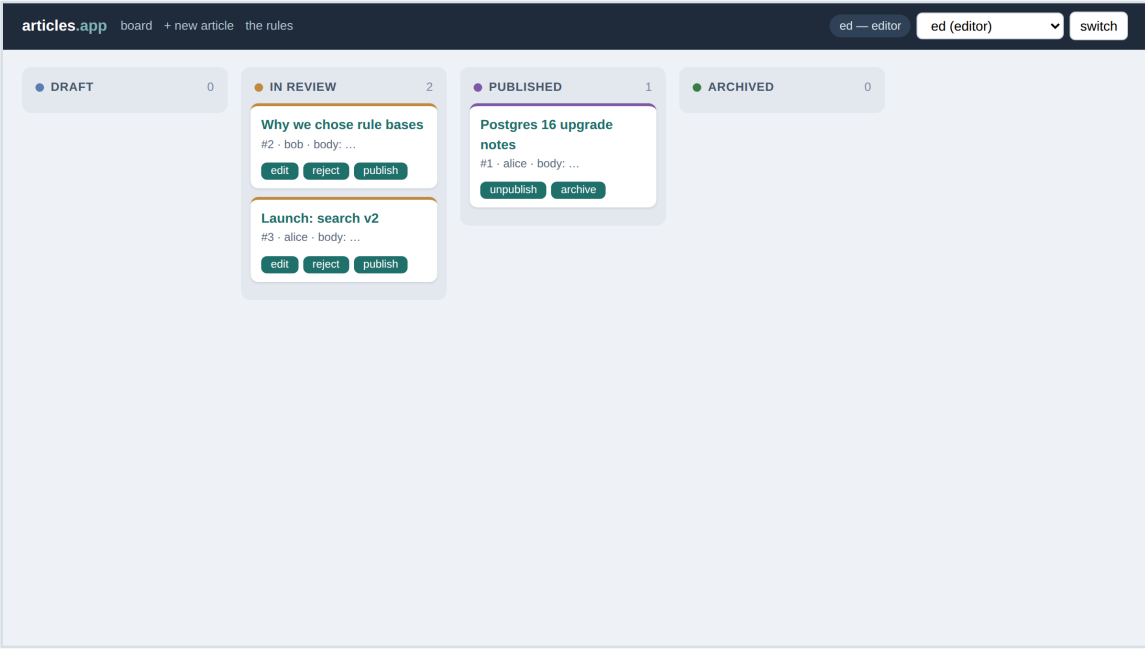
Three rules visible at once: delete locked by nothing_is_deleted, approve locked by no_self_approval, and the edit form warning “editing is refused right now by review_is_sealed” (TB-4: content sealed under review).

A support question answers itself: “why can’t I?” → the rule id and the stakeholder sentence it translates.

The rules page; and the same UI serving the CMS



/ui/rules renders rules.yaml — lifecycle, every rule with its ticket sentence, projections, assumptions. The program is its own documentation, in the app.



Generality, executable: the identical UI module serving the CMS (editor persona, publish/reject chips, drafts hidden). The CMS gained a browser UI without changing a line.

The phenomenology, kept honest (note 14)

Review latency is gone

0.18 s from edit to verdict, and the verdict is terminal: PASS = mergeable; FAIL = the situation + the rule, by name. Like a type checker for authorization — that also demands liveness, so “deny everything” cannot pass.

The gate is where the thinking happens

Writing `safety.yaml` forced the questions the rules let me fudge (“may staff work a task?”). $\forall$ -properties are the design review; the rules are just the design.

The unit of work is a sentence

Every edit was a ticket-shaped stanza. There was never a “now write the handler” moment — after the gate went green, building the app meant writing a seed list.

Predictions get falsified fast

“The containment deny is dead” — falsified in round 1, true in round 3, one fix later. The dead-rule check is a live diagnostic of the containment regime.

Bugs tests would miss

Both real holes were rule interactions. The solver considers the situation nobody writes a test for — task assigned to “anonymous” — and hands you the granting rule.

The app is a byproduct

UI, API, storage, named errors: derived. Zero lines of Flowdeck-specific code exist in the repository.

The sharp edges are the research — all enumerable, none a wall

Namespace surprise

Relational projections read like actor facts but live under resource.*
(resource.assigned_to_me). Load error caught it; naming needed a convention. Cost: one round.

Deny-order naming

When several denies match, denied_by is declaration-order. Named-denial probes must sit in the minimal state that triggers them. A future analyzer check could flag ambiguous probes.

Edit is blind to proposed values

Rules see an edit's current fields, not the incoming ones — “a title can never be emptied later” is not expressible; S3 guards creation only. Vocabulary gaps fail loud; semantic gaps fail silent. Sharpest edge found.

Assumptions are trusted, not proven

Nothing stops assignee: “ada” at runtime; an assignees-are-team-members assumption would be unchecked (only authorship is cross-verified mechanically today).

The vocabulary is a budget

Every boolean doubles the situation space: 276,480 → 34,560 after has_-opt-out. Exhaustive backend agreement is ~85 s of check.sh — RB2's known scaling edge.

The engine wants to be a package

Flowdeck path-shims ../rule-driven-cms. Fine for research, grating for product work.

Four services, one engine, every claim executable

4 : 1

services per engine — CMS, tickets, receivables, Flowdeck on 1,615 lines of domain-free Python (455 of them the web UI every rule base gains at once).

377 : 0

lines of YAML that ARE Flowdeck vs lines of app-specific code. CMS: 313. Receivables: 290. Tickets: 100.

70,656

situations checked exhaustively across the four services — runtime evaluation and the Z3 compilation agree on every one.

16 + 4

named findings across two sabotaged rule bases and the preserved round-2 draft; zero false passes; every finding names a rule or prints a situation.

Transfer cost per new domain (falsifier RB1) — one missing generic concept each, then flat:

- CMS → tickets: 0 engine lines. → receivables: ~100 (the concept: TIME — clock, date projections, feature clocks).
- → Flowdeck: ~60 (the concept: actor↔resource RELATIONS — tenancy, assignment) + the generic UI (~455, every service at once).
- Prediction on the table: a fifth domain (approvals/expenses) costs zero engine lines — time + relations + validation cover it.

What the analyzer asks of every change, in seconds: dead rules · stale assumptions · $\forall$ -safety · $\exists$ -possibility · lifecycle liveness + gated entries · frozen features with refusals by name.

Kept deliberately sharp

The projection boundary

Rules see finite booleans (`same_team`, `assigned_to_me`, `is_past_due`). Fuzzy matching, cross-item invariants (“no two claims share a reference”, quotas) live in clients — properly in store constraints (P9: the invariant→Postgres compiler that still doesn't exist, anywhere).

Edit can't see proposed values

New from the DX study: rules evaluate an edit against current fields, so “title can never be emptied” is unstatable. Known engine fix; until then this is the silent-gap class to watch (DX3).

No time-interleaving

Single-situation rules can't see races — the demoted author who is still `is_author`. Rung 2's job: the Quint temporal twin (RB4) stays on the ladder.

One entity per rule base

A kanban app fit in one. Comments/checklists would force relations between ruled entities — N rule bases + client joins, or engine work.

The engine is unproven Python

1,615 lines taken on faith. The production form is a verified engine (Cedar's Lean proofs, or our track-D Dafny→Go shape): one proof, then per-change analysis only.

What we learned the hard way — now enforced

1 · The spec is frozen for agents

Enforced mechanically — the analyzer takes --gate from a pinned directory; agents edit rules, never the gate. The taskboard's round-2 draft is gated forever.

3 · Gate strength beats NL steering

Same bug, model, prompt: the stronger gate turned a partial fix into the full fix (P4). Cheaper models are fine when the gate is strong.

5 · Translate / validate split

LLM translation of NL→rules is unsound — always followed by a sound solver step. Humans review rules sentence-by-sentence, never agent edits.

7 · Verify claims; record falsified predictions

The DEVLOG keeps the author's wrong prediction (round 1) next to the run that killed it. The checker corrected the author — again. That is the method working.

2 · Gates need both directions

Safety-only gates accept fixes that trade away liveness (“the safest system does nothing”). Every gate here = safety + possibility witnesses + frozen feature runs.

4 · Escalate to proofs

Rules (this deck) < temporal models < inductive < parameterized < liveness < hyperproperties. Climb only when the ticket needs time, relations, or computation.

6 · Counterexamples are the currency

Named and machine-readable: situation witnesses, blocking denies, denied_by 403s — now also greyed buttons and banners in the UI.

8 · Boundaries by construction

Bots are ordinary actors with no back door; the janitor is contained by default-deny, proven by S11 — after the solver showed the scope rule was dead.

Falsifiers on the table

Held so far

- RB1 (tickets land as rule diffs): held three times — imports 0 engine lines, receivables ~100 (time), Flowdeck ~60 (relations). Gate-language growth is flattening as predicted.
- DX verdict (note 14): for the policy-shaped core of a SaaS app, the loop beats conventional development on its own terms — faster iteration, terminal verdicts, interaction bugs caught, UI/API for free.

Next

- DX1 — an outside web developer ships a ruled app from tickets in under a day without reading the engine. DX2 — the same 7 tickets, implemented conventionally by a strong LLM: does it contain either interaction hole the gate caught here?
- DX3 / engine — close the edit-proposed-values gap (evaluate edits against incoming fields); add the ambiguous-probe warning.
- RB3 — ticket→rule-diff vs ticket→handler-code fidelity, measured. RB5 — agent repair on the sabotaged rule bases under the frozen gate.
- RB4 — the Quint temporal twin (stale-role races, clock skew). RB2 — symbolic backend-agreement when exhaustive hurts. P8 — the same rules as Cedar policies. P9 — the invariant→Postgres-constraint compiler nobody has built.

Parked (consciously, with reasons recorded)

Alloy (Z3 covers it) · Quint→Rust codegen (no tooling) · Kani until unbounded domains arrive · Verus (proxy-blocked; Dafny stands in).

The one-sentence takeaway

We built a working multi-tenant SaaS product out of seven tickets and 377 lines of rules — no handlers, no app UI code — and the solver reviewed every change in a fifth of a second, catching two authorization holes no test suite would have looked for.

Run it: `examples/taskboard/check.sh · python examples/taskboard/app.py → /ui`

Read it: `examples/taskboard/DEVLOG.md` (the raw journal) · `research/14-developer-experience.md`

The stack beneath: `research/13` (rules as the program) · 09–12 (proof escalation) · prototypes p1–p7